

Reviewer Pack — Algebraic K-Theory Rank Bounds on Communication Complexity o...

Entry d263bdd13ae5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 09:20:16 UTC

Algebraic K-Theory Rank Bounds on Communication Complexity of XOR via Symmetry Detection

Entry ID: d263bdd13ae5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 09:20:16 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Communication Complexity

Statement:

[‘For every Boolean function f with n inputs, the communication complexity $CC_XOR(f)$ for evaluating f at a random input is upper-bounded by $O(\alpha(n))$, where $\alpha(n)$ is the rank of the free algebra generated by the set of variables of f over the field with two elements.’, ‘Additionally, there exists an absolute constant $c > 0$ such that for all n and Boolean functions f , $CC_XOR(f) \geq cn^2/\text{rank}(F)$, where F is the algebraic K-theory group of the free algebra generated by the variables of f .’, ‘For a given Boolean function f , if $CC_XOR(f) = O(1)$, then $\text{rank}(F) \leq n$.’]

Rationale (proposer’s reasoning):

[‘The connection between Algebraic K-Theory and communication complexity is relatively underexplored. This conjecture suggests that the rank of the free algebra, which is a fundamental invariant in Algebraic K-Theory, could be related to the communication complexity of evaluating Boolean functions.’, ‘Symmetry detection, as a key problem in communication complexity, provides a natural context for this conjecture. If true, it would establish a new bridge between Algebraic K-Theory and computational complexity, offering potential insights into both fields.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 077f9537c38835ec

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Boolean function f with n inputs, if the estimated rank of the free algebra F is within 10% of $\alpha(n)$ and $CC_XOR(f) \leq O(\alpha(n))$, then support for the conjecture is provided.

Falsification occurs if any seed produces a rank estimation outside 90% confidence interval of $\alpha(n)$, or $CC_XOR(f) > O(\alpha(n))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic k-theory AND communication complexity - rank bounds AND XOR AND communication complexity - symmetry detection AND algebraic K-theory AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def free_algebra_rank(f):
        n = len(f)
        variables = set()
        for term in f:
            for var in term:
                if var.startswith('x'):
                    variables.add(var)
        num_vars = len(variables)
        return num_vars

    def communication_complexity(f):
        n = len(f)
        rank_F = free_algebra_rank(f)
        return n * math.log2(n) / rank_F

    def alpha(n):
```

```

    return 1.4426950408889634 # Approximation of  $\alpha(n)$  for large  $n$ 

instances_tested = 0
total_metric_value = 0
conjecture_holds = True
counterexample = ""

for _ in range(100): # Test with at least 30 different random seeds
    f = generate_boolean_function(random.randint(5, 40))
    rank_F = free_algebra_rank(f)
    cc_f = communication_complexity(f)

    if rank_F <= len(f):
        instances_tested += 1
        total_metric_value += cc_f

        if not (alpha(len(f)) * 0.9 <= cc_f <= alpha(len(f)) * 1.1):
            conjecture_holds = False
            counterexample = f"CC_XOR(f) out of bounds: {cc_f} for n={len(f)}"

return {
    "metric_name": "communication_complexity",
    "metric_value": total_metric_value / instances_tested if instances_tested > 0 else 0,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["instances_tested"] > 0) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r["instances_tested"] > 0) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={repr(counterexample)} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unreachable")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the conditions for supporting or falsifying the conjecture could not be determined. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12100
2	preregistration	ollama_remote	glm4:latest	0	0	6169
3	novelty	ollama_remote	glm4:latest	0	0	4569
4	novelty	ollama_remote	glm4:latest	0	0	9457
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15984
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10073
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9325
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8900
9	judge	ollama_remote	glm4:latest	0	0	12897

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89474 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/d263bdd13ae5.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/d263bdd13ae5.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/d263bdd13ae5.tar.gz* (if generated)